

aWebSite

Test Documentation & Deployment Guide

Project: D:\gitSpace\laWebSite

Framework: Django 5.2 + pytest-bdd 8.x + Playwright

Language: Python 3.11 (VPS) / 3.13 (Dev)

Generated: June 2026

1. Project Overview

aWebSite is a Django-based personal website with user authentication, blog posts, body weight tracking (with screenshot OCR), user management, and internationalization (English/Chinese).

Core Features

- User registration, login, logout, password management
- Blog post CRUD with pagination and i18n translation support
- Body weight tracking: upload scale screenshots, OCR extracts measurement data
- User management dashboard with IP/location tracking
- Django admin interface for content moderation
- Language switching (English / Simplified Chinese)
- Responsive UI with CSS styling

Architecture

Component	Technology	Notes
Backend	Django 5.2	SQLite database
Frontend	HTML + CSS + Django Templates	No JavaScript framework
OCR	winrt (Win) / Tesseract (Linux)	Free, local OCR
Vision Fallback	OpenRouter API	When local OCR fails
Tests	pytest-bdd + Playwright	BDD + browser automation
Production	VPS CentOS 7 (23.224.152.81:8000)	Python 3.11, no Docker

2. Deployment

2.1 VPS Info

Item	Value
IP	23.224.152.81
OS	CentOS 7.9 (1 CPU, 351MB RAM)
Python	/usr/local/python311/bin/python3 (3.11.11)
Django	5.2.15
Port	8000 (iptables opened)
DB	SQLite (db.sqlite3, SQLite 3.49.1)

2.2 Quick Deploy

From the local Windows machine (D:\gitSpace\laWebSite):

Step 1: Package the project

```
cd "D:/gitSpace/aWebSite" && tar czf /tmp/aweb.tar.gz --exclude=website --exclude=__pycache__ --exclude=.claude --exclude=*.pyc --exclude=.git .
```

Step 2: Transfer via SCP (password auth via SSH_ASKPASS)

```
scp -o StrictHostKeyChecking=no /tmp/aweb.tar.gz root@23.224.152.81:/root/
```

Step 3: Extract & migrate on VPS

```
export PATH=/usr/local/python311/bin:$PATH\nexport LD_LIBRARY_PATH="/usr/local/lib:/usr/lib64/openssl11:$LD_LIBRARY_PATH"\ncd /root && tar xzf aweb.tar.gz\npython3 manage.py makemigrations\npython3 manage.py migrate\npython3 manage.py collectstatic --noinput
```

Step 4: Start the server

```
pkill -f "manage.py" 2>/dev/null; sleep 1\nnohup python3 manage.py runserver 0.0.0.0:8000 > /tmp/django.log 2>&1 &
```

2.3 Dependencies

VPS Python packages: django, python-dotenv, requests, SpeechRecognition, Pillow, pytesseract. System packages: tesseract + chi_sim language data, openssl11, sqlite 3.49.1 (compiled from source).

■ **Note:** The VPS has limited resources (351MB RAM, 1 CPU). The dev server (manage.py runserver) is sufficient for personal use. For production, consider gunicorn + nginx.

3. Test Design Methodology

3.1 Approach: BDD + Playwright

Tests use **pytest-bdd** (Gherkin language) for human-readable scenario specifications combined with **Playwright** for browser automation. This gives us:

- Executable specifications — feature files are both documentation and tests
- Real browser rendering — tests run against Chromium via Playwright
- Isolation — each scenario gets a fresh browser context
- Live Django server — conftest.py auto-starts a dev server per session
- Django ORM access — test data creation via ORM (not UI) for speed

3.2 Architecture

test_dir/

```
■■■ conftest.py # All fixtures + ALL step definitions\n■ (Django setup, live server, Playwright\nbrowser, page, screenshots)\n■■■ test_scenarios.py # Loads all .feature files\n■■■ pytest.ini\n# Markers for test categorization\n■■■ features/ # Gherkin feature files (.feature)\n■ ■■■\nauthentication.feature (6 scenarios)\n■ ■■■\nposts.feature (6 scenarios)\n■ ■■■\nnavigation.feature (7 scenarios)\n■ ■■■\ndashboard.feature (6 scenarios)\n■ ■■■\nweight-data.feature (5 scenarios)\n■ ■■■\nadmin.feature (5 scenarios)\n■■■ steps/ #\n(deprecated, steps moved to conftest)\n■■■ conftest.py\n■■■ ... (old step files kept for\nreference)
```

3.3 Why Steps in conftest.py

In pytest-bdd 8.x, **@given/@when/@then decorators create pytest fixtures** which must be registered in conftest.py for proper discovery. Step definitions in external modules are not found by pytest-bdd's fixture scanning. All ~60 step definitions are therefore in test_dir/conftest.py.

3.4 Key Design Decisions

Decision	Rationale
ORM for test data	Speed: create users/posts via DB, not browser UI
Fresh context per test	Isolation: no state leakage between scenarios
Session-scoped server	Performance: start Django once per test session
Screenshot on failure	Debug: auto-capture page state when test fails
@given/@when/@then	Clarity: explicit step types match Gherkin grammar

4. Test Coverage

4.1 Feature Coverage Matrix

Feature	Scenarios	Covers	Status
Authentication	6	Register, duplicate user, login, invalid login, logout, unauthenticated access	■ 1/6 pass
Blog Posts	6	Empty list, create post, detail view, ordering, guest restriction, pagination	■ 1/6 pass
Navigation/18n	7	Guest/authed nav, lang switcher presence, Chinese/English content assistance	■ 0/7 pass
Dashboard	6	Info cards, user table, row highlight, multi-user, IP/location column, pagination	■ 0/6 pass
Weight Data	5	Upload form, image upload, records list, guest redirect, column display	■ 0/5 pass
Admin	5	Admin login, authed access, user CRUD, edit form, guest restriction	■ 0/5 pass

4.2 Total Stats

Metric	Value
Feature files	6
Total scenarios	35
Passing	3 (9%)
Failing	32 (91%) — mainly assertion mismatches vs actual site
Step definitions	~60 (all in conftest.py)
Framework status	■ All steps discovered, server starts, browser runs

■ **Note:** The 32 failing tests fail because the Gherkin assertions expect specific text/elements that differ from the actual live site. The test infrastructure (Django server, Playwright, step discovery) is fully operational. Each failure identifies a discrepancy between the specification and the actual UI — these are valuable for test-driven refinement.

4.3 Coverage Gaps

- No post translation/transcription tests
- No comment tests (list, receive)
- No password reset / email notification tests
- No responsive/mobile viewport tests
- No performance or load tests
- No API-level tests (all tests are through the browser)

5. Running Tests

5.1 Prerequisites

- Python 3.10+ with project venv activated
- Playwright browsers installed: playwright install chromium
- Django project files in place (D:\gitSpace\AWebSite)
- Python packages: pytest, pytest-bdd, playwright, Pillow

5.2 Run All Tests

```
cd D:\gitSpace\AWebSite\nwebsite\Scripts\python.exe -m pytest test_dir/test_scenarios.py -v
```

5.3 Run Specific Features

```
# Run only authentication tests\nwebsite\Scripts\python.exe -m pytest test_dir/test_scenarios.py  
-k "register or login or logout" -v
```

```
# Run only smoke tests (with marker)\nwebsite\Scripts\python.exe -m pytest -m smoke -v
```

5.4 Run a Single Scenario

```
website\Scripts\python.exe -m pytest  
test_dir/test_scenarios.py::test_register_a_new_account_successfully -v --tb=short
```

5.5 Command-Line Options

Flag	Effect
-v	Verbose output (show each test name)
--tb=short	Short traceback (focus on failure message)
--tb=long	Full traceback with all context
-k "keyword"	Filter tests by name keyword
-m "smoke"	Run only tests with @pytest.mark.smoke
-x	Stop after first failure
--headed	Run browser in headed mode (watch the test)
--slowmo 500	Slow down Playwright by 500ms between actions
-s	Print stdout from tests (no capture)

5.6 Debugging Failing Tests

When a test fails:

- A screenshot is saved to test_dir/screenshots/ automatically
- Use --tb=long to see the full error traceback
- Add --headed to watch the browser during test execution
- Check /tmp/django.log on VPS (or local server log) for Django errors
- Run interactively in Python: from test_dir.conftest import *; then call steps

5.7 Quick Verification

```
# Quick check: framework works (1 test, ~3 seconds)\nwebsite\\Scripts\\python.exe -m pytest  
test_dir/test_scenarios.py::test_dashboard_requires_authentication -v
```